
IOCP 서버 네트워크 모듈

2026.02.19

조성찬

목차

1. 개요

... 3

1.1. 특징

2. 전체 설명

..... 5

2.1. 전체 조망

2.2. 전체 시스템 구성

2.3. 전체 동작 방식

3. 동작 설명

..... 9

3.1. Accept 스레드

3.2. 워커 스레드

3.3. 세션

3.4. 패킷

3.5. TLS 오브젝트 풀

4. 기타 특이사항

..... 18

4.1. 사용법

4.2. 테스트 환경

1. 개요

네트워크를 사용하는 어플리케이션을 효율적으로 구현하기 위해서는 네트워크 처리의 추상화가 반드시 필요하며, 연결 수가 증가할수록 효율적인 I/O 처리 방식이 필수적이다.

본 모듈은 Windows에서 제공하는 고성능 비동기 I/O모델인 IOCP를 사용한다. IOCP는 커널 레벨에서 I/O 완료 이벤트를 큐에 등록하고, 어플리케이션은 해당 이벤트만 처리함으로써 불필요한 소켓 검사를 제거한다. 또한, 스레드 풀과 자연스럽게 연동되어 CPU 자원을 효율적으로 활용하며, 수천~수만 개의 동시 연결을 안정적으로 처리할 수 있다.

IOCP 기반의 네트워크 모듈은 대규모 서버 어플리케이션에 적합하며, 안정성과 성능을 동시에 확보할 수 있는 구조적 장점을 가진다. 본 네트워크 모듈은 이러한 IOCP의 특성을 활용하고, Accept스레드, 워커스레드, 세션 관리, 패킷 직렬화 버퍼, TLS 오브젝트 풀, 링버퍼 및 락-프리 큐로 구성된 버퍼 등의 핵심 구성 요소들을 설계 및 구현함으로써 대규모 네트워크 환경에서의 효율적인 처리를 도모한다.

1.1. 특징

IOCP를 기반으로 대규모 동시 접속 환경에서 비동기 I/O를 안전하게 처리하여 장기 가동이 가능하다. 또한 네트워크 애플리케이션 개발 시 반복적으로 발생할 수 있는 복잡한 처리 과정을 추상화하여 재사용 가능한 구조를 제공한다.

[특징 1] 추상화 (생산성 향상)

어플리케이션 개발자가 네트워크의 복잡함에 매몰되지 않도록 사용자 친화적인 인터페이스를 제공한다.

- **복잡성 은닉:** 소켓 생성, 연결, 송수신 처리 등은 직접 구현 및 검증 시 많은 복잡성을 수반하는데, 이러한 어려움을 은닉한다.
- **이벤트 기반 설계:** Accept, Send, Recv, Close 등의 핵심 이벤트를 가상 함수를 기반으로 설계하여 상위 어플리케이션과의 의존성을 분리한다.
- **의존성 분리:** 네트워크 라이브러리와 상위 로직 사이의 결합도를 낮춰, 어플리케이션 개발 시에 작업자가 핵심 로직에 집중할 수 있도록 도움을 준다.

[특징 2] 확장성 (유지보수 용이성)

단일 프로젝트용이 아닌, 다양한 서비스에 즉시 투입할 수 있는 견고한 구조의 범용 모듈을 지향한다.

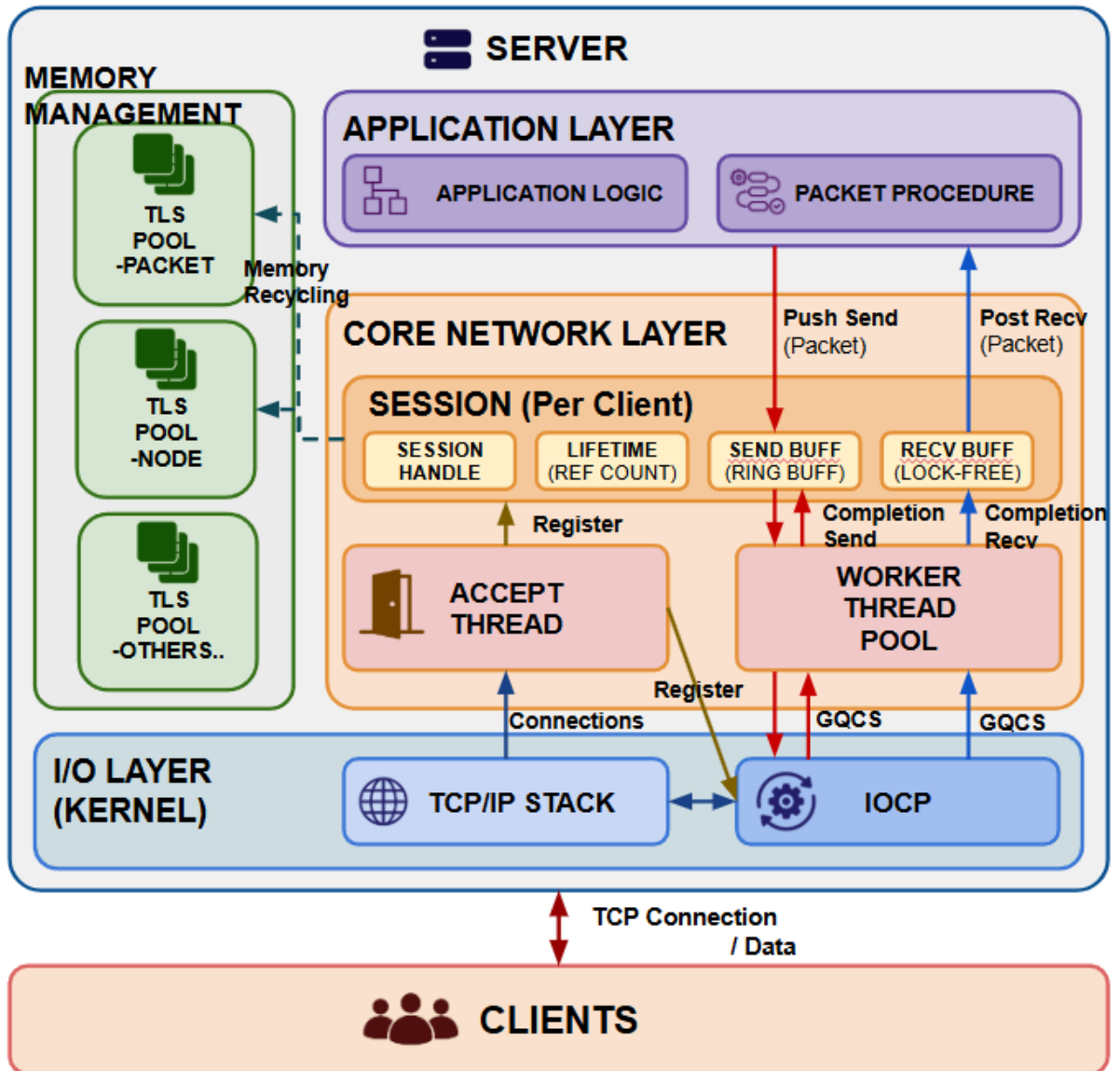
- **범용성:** 게임 서버, 채팅 서버, 실시간 메시징 시스템 등 다양한 네트워크 애플리케이션에 적용 가능한 범용 구조를 제공한다.
- **기초 방어:** XOR 기반의 간단한 난독화와 유효한 수신 패킷 검사를 통해 잘못된 패킷 주입으로 인한 서버 다운을 방지하는 최소한의 방어를 포함한다.
- **재사용:** 재사용 가능한 구조를 제공하여 향후 다양한 네트워크 어플리케이션 개발에서 생산성을 높이고, 유지보수 비용을 절감한다.

[특징 3] 대규모 연결 처리 적합성

- **송신 제로-카피:** 송신 시의 커널 버퍼와 유저 버퍼 사이의 복사를 생략하는 Zero-copy 옵션을 제공하여 불필요한 메모리 복사를 줄여 처리량을 높인다.
- **대규모 연결:** 대규모 연결 환경에서 세션 객체를 통해 각 연결을 독립적으로 관리하며, 수명 관리·버퍼 관리·송수신 과정을 체계적으로 처리한다.
- **세션 간 분산 처리:** IOCP 완료 통지 구조를 통해 세션 간의 실행 흐름을 완전 독립시켜 전체 워커 스레드 풀에 분산 처리되므로, 특정 연결의 처리 지연이 다른 연결의 대기 시간으로 전이되지 않는다.

2. 전체 설명

2.1. 전체 조망



2.2. 전체 시스템 구성

2.2.1. Accept 스레드

클라이언트의 연결 요청(connect)을 대기하고 수락한다. TCP 연결 수립 후 최소한의 로직 처리를 수행하며, 연결 요청과 시스템을 분리한다.

2.2.2. 워커 스레드

I/O 완료 통지를 통해 실제 패킷을 처리하는 모듈의 핵심 요소이다.

2.2.2.1. 수신 완료 처리

OS 커널 버퍼에 들어온 데이터를 세션의 수신 버퍼로 복사하고, 완성된 패킷 단위로 분리하여 상위 어플리케이션으로 전달한다.

2.2.2.2. 송신 완료 처리

전송할 데이터를 세션의 송신 버퍼에 쌓고, 커널에 송신 요청을 한다.

2.2.3. 세션

클라이언트와 네트워크 모듈 간의 연결 정보를 담는다.

2.2.3.1. 세션 핸들

각 연결을 식별하기 위한 고유 식별자이다. 본 모듈은 직접적인 소켓 핸들을 노출하지 않고 임의의 핸들을 상위 어플리케이션에게 제공한다.

2.2.3.2. 세션 수명 관리

연결 끊김 시 진행 중인 I/O를 모두 완료한 후 안전하게 해제한다. 본 모듈은 참조 카운트를 통해 세션 수명을 관리한다.

2.2.3.3. 세션 버퍼

본 모듈의 세션은 세 종류의 버퍼를 활용해 계층간의 흐름 차이를 완충한다.

2.2.4. 패킷 (직렬화 버퍼)

가변 길이 데이터를 네트워크로 전송 가능한 형태를 제공한다.

본 모듈에서 전달되는 완성된 데이터의 최소 단위이다.

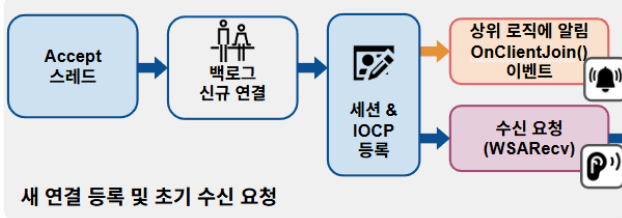
2.2.5. TLS 오브젝트 풀

자주 생성/해제되는 객체에 대하여 각 스레드마다 전용 풀을 두어 스레드 간 경합 없이 메모리를 고속으로 할당/해제한다.

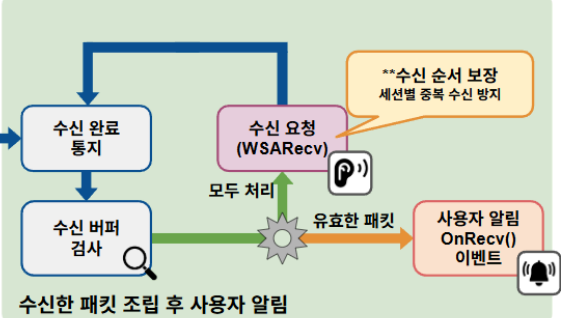
2.3. 전체 동작 방식

전체 동작 방식 요약

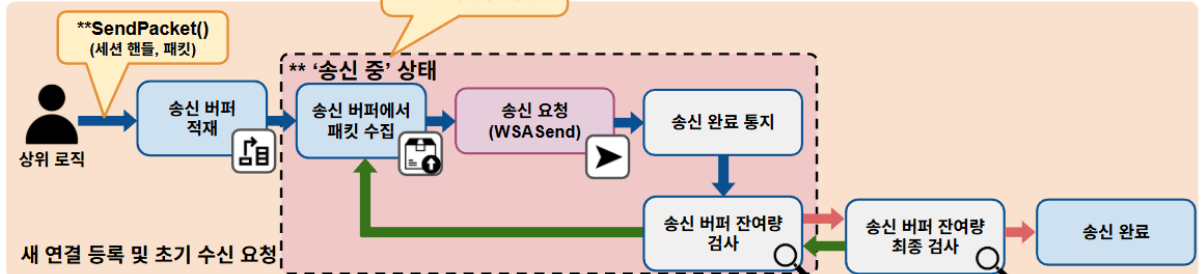
[요약 1] **2.3.1. 연결**



[요약 2] **2.3.2. 수신**



[요약 3] **2.3.3. 송신**



2.3.1. 연결

[클라이언트]	[Accept 스레드]	[IOCP]	[세션]	[상위 어플리케이션]
	---- 접속 요청 ---->			
	--- (1)accept() 수락			
	--- (2)소켓 등록 ---->			
	--- (3)세션 등록 ----->			
	----- (4) OnClientJoin() 이벤트 통지 ----->			
	----- (5) WSARecv() 호출 (최초 수신 요청)			

- **(1) Accept:** Accept 스레드는 백로그에 담긴 새로운 연결의 접속을 수락한다.
- **(2, 3) 등록:** 생성된 소켓 핸들을 IOCP에 등록하고, 세션에 등록한다.
- **(4) 이벤트 통지:** 상위 어플리케이션 계층에 `OnClientJoin()` 을 호출하여 새로운 연결이 성립되었음을 알린다.
- **(5) 최초 수신 요청:** 세션 등록 직후, 데이터 유입을 대기하기 위해 `WSARecv()` 를 호출하여 첫 번째 비동기 수신 요청을 수행한다.

2.3.2. 수신

```

[ 워커 스레드 ]           [ 세션 (수신 버퍼) ]           [ 상위 어플리케이션 ]
|
| -- (1) GQCS 통지 확인 (수신) - - -> |
|                                     |
|< - (2) 버퍼 데이터 읽기 (패킷 조립) - |
|
| -- (3) 패킷 완성시 OnRecv() 이벤트 통지 ----->|
|
| -- (4,5) WSARcv() 호출 (수신 재요청)

```

- (1) 수신 완료 통지: 수신 완료를 `GetQueuedCompletionStatus()` 를 통해 통지받는다.
- (2) 버퍼 데이터 읽기: 수신 버퍼 내 데이터의 헤더를 확인하여 패킷의 유효성을 검사한다.
- (3) 이벤트 통지: 완성된 패킷을 `OnRecv()` 이벤트를 통해 상위 어플리케이션에 전달한다.
- (4) 순서 보장: 동일 세션에 대해 TCP 스트림이 여러 스레드에서 뒤섞이며 처리되는 것을 방지하기 위해 `WSARecv()` 가 동시에 여러 번 호출되지 않도록 한다.
- (5) 수신 재요청: 수신 버퍼 내의 데이터 처리가 완료되면 다시 `WSARecv()` 를 호출하여 다음 수신을 요청한다.

2.3.3. 송신

```

[ 상위 어플리케이션 ]           [ 세션 ( 송신 버퍼 ) ]           [ 워커 스레드 ]
|
|
(1) SendPacket() 호출
|-----[ 패킷 큐잉 ]-----> |
|                               |
(2) '송신 중' 플래그 확인      |
[ IF False ] -> True 설정. 비동기 송신 요청 ----- >|
|                               |
|                               |
|<---[버퍼에서 패킷들 추출]-----|
|                               |
|                               (3) WSASend() 호출
|                               |
|                               |
|                               |
|                               (4) 송신 완료 통지 발생
|---[잔여 패킷 확인/재송신]--- |
|
|
'송신 중' 플래그 해제

```

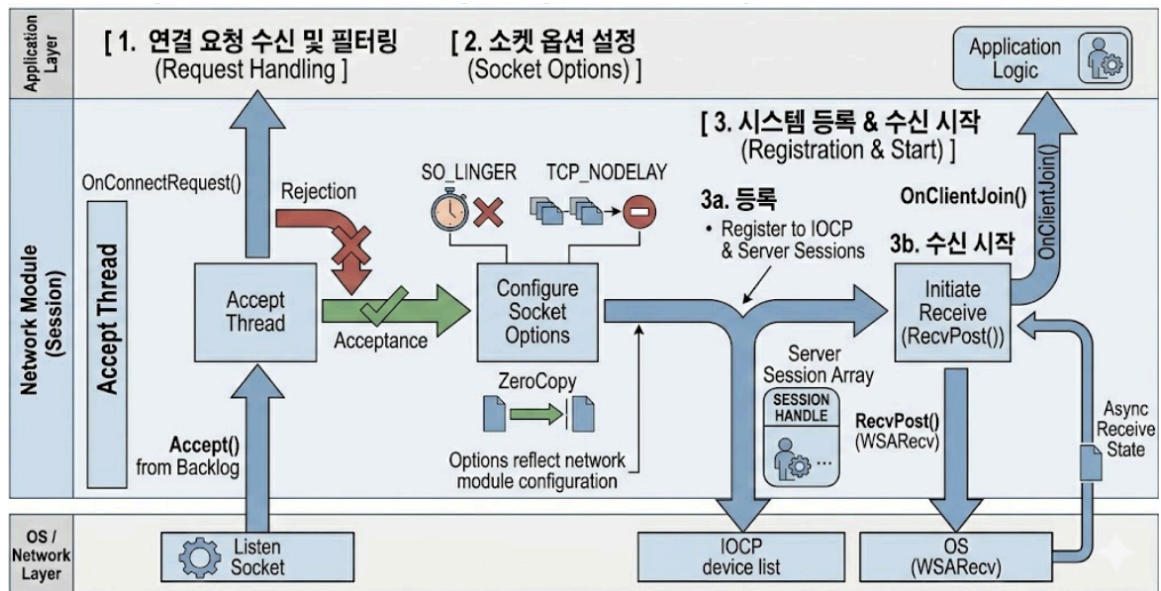
- (1) 송신 요청 큐잉: 상위 어플리케이션이 `SendPacket(SessionHandle, Packet*)`을 호출하면, 데이터는 즉시 전송되지 않고 세션별 송신 큐에 적재된다.
- (2) 비동기 송신 신호: 현재 해당 세션에서 진행 중인 송신(`WSASend`)이 없다면 비동기 송신을 요청한다.
- (3) 비동기 송신: 큐에 쌓인 패킷들을 모아 한 번에 송신 요청한다.
- (4) 송신 완료 통지: 송신 완료를 `GetQueuedCompletionStatus()`를 통해 통지받는다. 완료 통지에서는 송신 버퍼에 남은 데이터가 있는지 확인하고 있다면 다시 송신을 시작한다. 없다면 송신을 종료한다.

송신 상태 확인에 사용되는 '송신 중 플래그'는 현재 세션에서 진행 중인 송신이 있다면 중복으로 송신할 수 없도록 제한하는 역할을 한다. IOCP 자체는 송신을 중첩시킬 수 있지만 이 모듈은 송신 순서와 버퍼 안전성을 보장해야 하므로 한 소켓에 대해 송신 완료 통지가 돌아오기 전까지는 다음 송신을 걸지 않는 방식을 채택한다.

3. 동작 설명

본 장에서는 2.2. 전체 시스템 구성에 나열된 각 핵심 요소들의 내부 동작과 역할을 설명한다.

3.1. Accept 스레드



Accept 스레드는 서버의 초기 연결 관문을 담당하며, 새로운 연결을 IOCP와 세션에 등록하는 역할을 수행한다.

Accept 스레드가 백로그에 대기 중인 클라이언트 연결을 가져오면 어플리케이션에게 OnConnectRequest()를 통해 알리고, 연결을 수락할지 결정하도록 요구하여 불필요한 리소스 낭비를 방지한다.

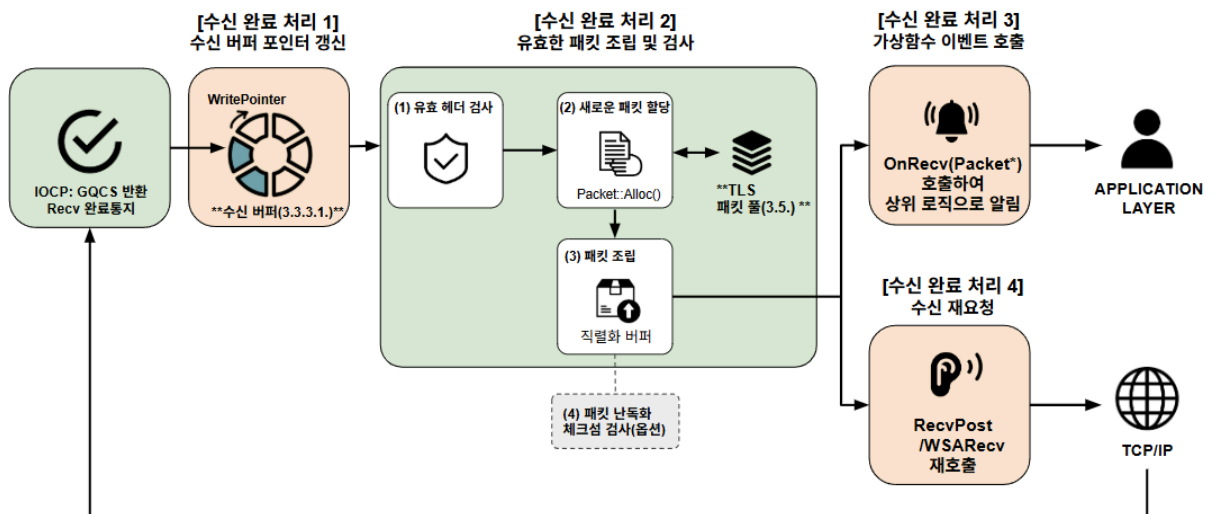
연결이 수락되면 네트워크 모듈 가동시에 설정된 SO_LINGER, TCP_NODELAY, ZeroCopy 등의 옵션을 해당 연결에 반영한 후 IOCP 장치 리스트와 서버의 세션에 등록한다.

이후 어플리케이션에게 OnClientJoin()을 통해 새로운 세션을 알리고 수신을 요청한다.

3.2. 워커 스레드

워커 스레드는 완료 통지가 발생하면 해당 소켓과 연결된 세션 객체를 찾아 후속 조치를 취한다.

3.2.1. 수신 완료 처리



수신 완료 처리에서는 커널이 세션의 수신 버퍼에 채워놓은 데이터를 어플리케이션으로 전달한다.

워커스레드의 수신 처리에서는 다음과 같은 순서로 작업을 진행한다.

[수신 완료 처리 1] 수신 버퍼 포인터 갱신

실제 수신된 바이트(NumberOfBytesTransferred) 만큼 **수신 링 버퍼(3.3.3.1.)**의 쓰기 포인터를 이동시킨다.

[수신 완료 처리 2] 유효 패킷 검사 및 조립

수신 링 버퍼에 쌓인 데이터가 최소 패킷 헤더 크기 이상인지 확인한다. 헤더가 완성되었다면 헤더에 적힌 길이만큼 데이터가 존재하는지 확인하고, 존재한다면 **패킷 직렬화 버퍼(3.4.1.)**에 복사하여 완성된 패킷 객체를 만든다.

이 과정에서 난독화 옵션을 활성화한 상태라면 해독을 진행하며, 패킷이 유효한지를 검사한다.

[수신 완료 처리 3] 가상 함수 이벤트 호출

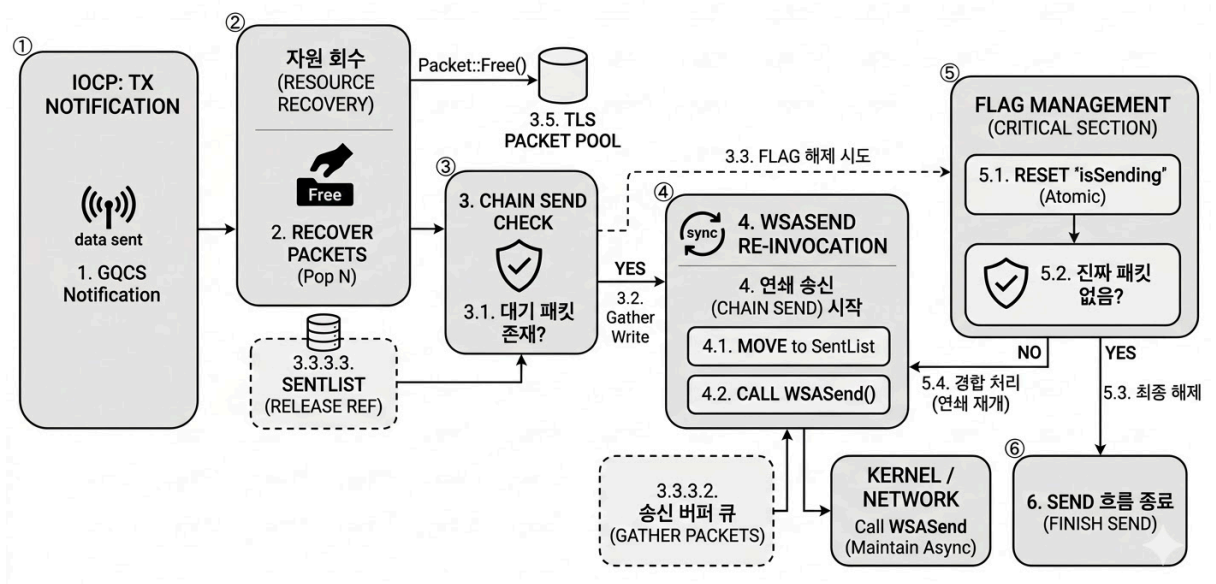
완성된 패킷을 `OnRecv(Packet*)` 가상 함수를 통해 넘겨 상위 어플리케이션 로직을 수행하게 한다.

[수신 완료 처리 4] 수신 재요청

데이터 처리가 끝나면 다시 다음 데이터를 받을 수 있도록 `RecvPost/WSARecv`를 호출하여 비동기 수신 상태를 유지한다.

3.2.2. 송신 완료 처리

CONCEPTUAL PIPELINE: SEND COMPLETION & CHAIN SEND (SIMPLIFIED)



송신 완료 처리에서는 사용된 패킷을 정리하고 다음 송신을 준비한다.

워커스레드의 송신 처리에는 다음과 같은 작업들을 포함한다.

[송신 완료 처리 1] 송신된 패킷 회수

송신된 패킷 리스트(3.3.3.3.)에 보관 중이던 패킷들의 참조 카운트를 감소시킨다. 카운트가 0이 된 패킷은 풀로 반납된다.

[송신 완료 처리 2] 연쇄 송신

송신 중에 송신 버퍼(3.3.3.2.)에 새로 쌓인 패킷이 있는지 확인한다. 만약 대기 중인 패킷이 있다면 다시 송신된 패킷 리스트로 옮기고 WSASend를 호출하여 연쇄적인 송신 흐름을 만들어낸다.

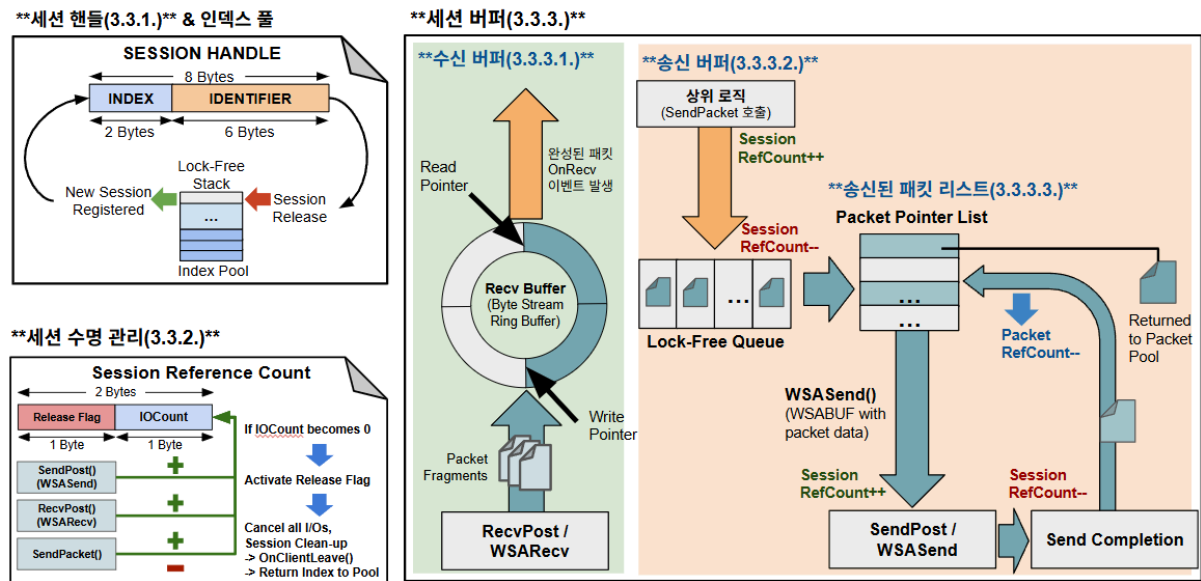
[송신 완료 처리 3] '송신 중' 플래그 관리

더 이상 송신할 패킷이 없다면 '송신 중' 플래그를 해제하여 나중에 `SendPacket()`이 호출될 때 다시 송신 흐름이 시작될 수 있도록 한다.

단, '송신 중' 플래그를 해제하기 직전에 또 다른 스레드에서 송신 버퍼(3.3.3.2.)에 패킷을 적재하고 송신에 실패하면 이 잔여 패킷은 영원히 송신되지 않을 수 있다. '송신 중' 플래그를 초기화한 직후에는 다시 한 번 송신 버퍼를 확인하여 진짜 흐름을 종료해도 괜찮은지 판단하는 방법으로 이를 방지한다.

3.3. 세션

3.3. Session Management System (세션 관리 시스템)



3.3.1. 세션 핸들

세션 핸들은 세션을 식별하는 8바이트 정수이다. 어플리케이션은 세션 핸들을 통해 네트워크 모듈에게 작업을 요청할 수 있다.

세션 핸들의 상위 2바이트는 세션 객체의 위치(인덱스)이다. 하위 6바이트는 세션의 유일한 식별자이다.

SESSION HANDLE (8 BYTES)

INDEX (UPPER 2 BYTES)	IDENTIFIER (LOWER 6 BYTES)
----------------------------	---------------------------------

본 모듈의 세션들은 배열로 관리된다. 배열의 길이는 네트워크 모듈 가동시에 설정된 최대 세션 수이다.

세션을 사용할 때에는 세션 핸들의 상위 2바이트에 쓰인 인덱스를 통해 빠르게 접근한다. 사용 가능한 인덱스가 2바이트이기 때문에 본 모듈에서 설정 가능한 최대 세션 수는 65535개 이다.

인덱스는 스택에 풀링되어있다. 새로운 세션이 등록되면 스택에서 하나의 인덱스가 할당되며, 세션이 해제되면 사용하던 인덱스를 스택으로 반환한다. 이러한 과정을 통해 하나의 인덱스는 하나의 세션에만 할당될 수 있으며, 세션 정보 공간을 재사용할 수 있도록 유도한다. 본 모듈은 이러한 인덱스의 관리에 락-프리 스택을 사용하고 있다.

3.3.2. 세션 수명 관리

세션의 수명은 참조 카운트로 관리된다.

참조 카운트의 상위 1바이트는 해제 플래그로, 해당 세션이 해제 처리에 들어갔음을 의미하는 플래그이다. 이 해제 플래그가 활성화되면 해당 세션에 영향을 주는 작업이나 I/O를 중단한다.

참조 카운트에 영향을 주는 동작에는 다음이 있다.

- 송신 요청 (`SendPost`)
- 수신 요청 (`RecvPost`)
- 어플리케이션의 패킷 송신 요청(`SendPacket`)

이러한 동작들은 시작 시에 참조 카운트를 확보하고, 작업을 완료하면 다시 감소시킨다.

카운트를 0으로 만든 주체는 세션의 해제 플래그를 활성화하고 세션의 해제를 진행한다.

세션이 해제되면 어플리케이션에게 `OnClientLeave()`를 통해 알리고, 세션이 사용하던 인덱스를 인덱스 풀에 돌려놓음으로써 재사용 준비를 마친다.

3.3.3. 세션 버퍼

세션은 각 계층 간의 흐름 차이를 극복하기 위해 내부에 세 종류의 버퍼를 가지고 있다.

3.3.3.1. 수신 버퍼

수신 버퍼는 네트워크 모듈이 어플리케이션에게 완성된 패킷 단위로 전달할 수 있도록 완충시키는 역할을 한다.

본 모듈에서의 수신 버퍼는 바이트 스트림 링버퍼로 구현되었다.

이 버퍼의 빈 공간은 세션 내부 `RecvPost()` 함수에서 `WSARecv`가 호출될 때 `WSABUF` 인자로 주어진다. 인자로 주어진 공간은 클수록 `syscall` 호출 당 많은 데이터를 가져올 수 있지만 그만큼의 페이지를 잠궈야 하기 때문에(페이지가 디스크로 나가서는 안됨) 성능에 영향을 줄 수 있다.

수신이 완료되면 수신한 데이터의 양 만큼 링 버퍼의 쓰기 포인터를 옮긴다. 이후 이 버퍼에서 완성된 데이터를 순서대로 꺼내어 어플리케이션으로 전달한다.

3.3.3.2. 송신 버퍼

송신 버퍼는 어플리케이션 측의 여러 스레드에서 **SendPacket()**을 통해 송신을 요청한 패킷을 자유롭게 던져놓는 공간으로, 어플리케이션과 네트워크 모듈 사이의 비동기 완충 지대이다.

본 모듈에서의 송신 버퍼는 락-프리 큐로 구현되었다.

SendPacket() 함수로 전달된 패킷은 참조 카운트를 증가시킨 후 송신 버퍼에 잠시 보관되었다가, 세션 내부 **SendPost()** 함수가 호출되면 이후 기술될 송신된 패킷 리스트로 옮겨진다.

3.3.3.3. 송신된 패킷 리스트

송신된 패킷 리스트는 단순히 패킷들의 포인터를 저장하는 공간이다.

이 버퍼는 세션 내부 **SendPost()** 함수가 호출되면 송신 버퍼에 담긴 패킷들을 옮겨 닳고, **WSASend()**가 호출될 때 옮겨진 패킷들의 데이터들을 **WSABUF** 인자로 넘긴다.

송신이 완료되면 이 버퍼에 담긴 패킷들의 참조 카운트를 감소시키고, 참조 카운트가 0이 된 패킷들을 반환한다.

3.4. 패킷

패킷은 본 네트워크 모듈이 데이터를 주고 받는 최소 단위이다. 이 구성 요소는 직렬화 버퍼를 포함한 객체로 구현되었다.

패킷은 빈번한 메모리 할당으로 인한 성능 저하를 방지하기 위해 **TLS** 오브젝트 풀(3.5.)을 사용한다. 상위 어플리케이션은 **Packet::Alloc()** 과 **Packet::Free()** 함수를 통해 패킷 풀에서 패킷의 할당과 해제를 할 수 있으며, 이를 자유롭게 사용할 수 있다.

3.4.1. 직렬화 버퍼

패킷 내에는 가변 길이를 수용 가능한 직렬화 버퍼가 포함되어 있다. 직렬화 버퍼는 다양한 자료형을 메모리상에 연속적인 바이트 배열로 변환하여 배치하여 최종적으로는 **WSASend** 시 별도의 가공 없이 즉시 전송 가능한 상태를 만든다.

직렬화 버퍼는 안전하게 데이터를 기록하고 읽을 수 있도록 두 가지 인터페이스를 제공한다.

[인터페이스 1] 스트림 연산자 오버로딩 (<<, >>)

C++의 **iostream**과 유사한 문법을 제공한다.

- << 연산자 (Push): 변수를 패킷에 순서대로 삽입한다. 내부적으로 데이터 타입의 크기를 계산해 Write 포인터를 자동으로 이동시킨다.
- >> 연산자 (Pop): 패킷 버퍼로부터 변수 타입만큼 데이터를 읽어와 변수에 저장한다. Read 포인터를 자동으로 이동시킨다.

[인터페이스 2] 직접 데이터 복사 (PutData, GetData)

연산자 오버로딩으로 처리하기 어려운 가변 길이의 바이너리 데이터나 구조체를 직접 다룰 수 있다.

- PutData(const char* src, int len): len 바이트만큼의 메모리를 패킷의 현재 위치에 복사한다.
- GetData(char* dst, int len): 패킷 버퍼에서 len 바이트만큼 데이터를 읽어 대상 주소(dst)로 복사한다.

다음 코드는 상위 어플리케이션에서 패킷의 직렬화 버퍼를 사용한 예시이다.

```
WORD messageLen;
WCHAR message[MSG_LENGTH];
unsigned long long accNo;

// 패킷에서 데이터 Pop
*packet >> accNo >> messageLen;
packet->GetData((char*)message, sizeof(WCHAR) * messageLen);
if (messageLen > MSG_LENGTH)
{
    server->Disconnect(sessionHandle);
    LOG(L"DetectAttack_MSG", LEVEL_DEBUG, L"Detected attack - MSG_LENGTH:
    %d", messageLen);
}

// 풀에서 새로운 패킷 할당
Packet* resPacket = Packet::Alloc();

// 패킷에 데이터 Push
*resPacket << (WORD)PACKET_CS_CHAT_RES_MESSAGE;
*resPacket << player->AccNo;
resPacket->PutData((char*)player->ID, sizeof(WCHAR) * ID_LENGTH);
resPacket->PutData((char*)player->Nickname, sizeof(WCHAR) * NICKNAME_LENGTH);
*resPacket << messageLen;
resPacket->PutData((char*)message, sizeof(WCHAR) * messageLen);

SendAroundSector(player, resPacket, true);

// 풀에 패킷 반환
Packet::Free(resPacket);
player->lastRecv = currentTime;
```

3.4.2. 수명 관리

네트워크 모듈 내에서 통신을 위해 사용되는 패킷들은 생성부터 반환까지 참조 카운트를 통해 관리된다.
이들은 NetAlloc()

브로드캐스트 최적화

만약 여러 세션에 동일한 패킷을 브로드캐스트 하는 경우에는 패킷을 매번 복사하지 않고, 하나의 패킷 포인터를 공유한다. 패킷이 각 세션의 송신 버퍼에 들어갈 때마다 참조 카운트는 증가하며, 송신이 완료될 때마다 참조 카운트가 감소한다.

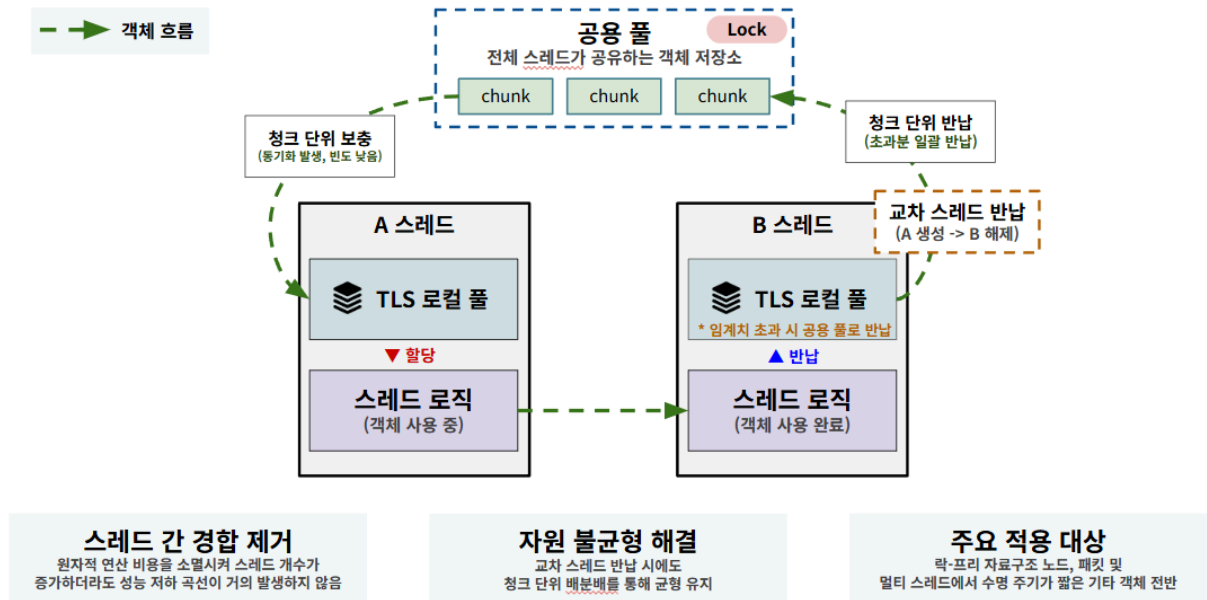
반환

참조 카운트가 0이 되는 순간 해당 패킷은 다시 패킷 풀로 반환되어 다른 용도로 재사용된다.

3.5. TLS 오브젝트 풀

3.5.1. 개념

TLS 오브젝트 풀은 일반적인 오브젝트 풀과는 달리, 각 스레드가 특정 객체(A)에 대해 자신만의 독립적인 로컬 풀(TLS 공간)을 가진다.



할당/반환: 스레드가 객체를 필요로 할 때 자신의 TLS 로컬 풀에서 락없이 객체를 꺼내 사용한다.

청크(노드 덩치) 단위 보충: 로컬 풀이 비어있으면, 전체 스레드가 공유하는 '공용 풀'에서 객체를 청크 단위로 한 번에 가져와 로컬 풀을 채운다.

3.5.2. 도입 배경

TLS 오브젝트 풀은 본래 락-프리 자료구조의 노드 풀로 사용하기 위해 제안되었다.

- **노드 힙 할당 위험성:** 락-프리 로직 특성상 힙에서 노드를 할당/해제하면 페이지 해제 시점이 불확실하기 때문에 다른 스레드에서 이미 해제된 메모리 주소를 참조하여 접근 위반을 발생시킬 수 있다.
- **독립 풀 생성의 비효율성 완화:** N개의 세션이 각각 락-프리 큐(송신 버퍼)를 가지는데, 락-프리 큐마다 전용 노드 풀을 가진다면 극심한 메모리 낭비가 발생하기 때문에 동일한 노드를 쓴다면 풀을 공유할 필요가 있다.
- **공유 풀의 병목:** 여러 스레드가 하나의 공유 풀에 동시 접근하면 필연적으로 병목이 발생한다. 이 병목 구간을 TLS로 분리하여 경합을 원천 차단한다.

3.5.3. 특징 및 장점

- **스레드간 경합 제거:** 할당과 반환의 대부분이 스레드 내부(TLS)에서 독립적으로 처리되므로 락이나 원자적 연산에 의한 경합 비용이 거의 제거된다.
- **확장성:** 경합이 없기 때문에 스레드 개수가 늘어나더라도 성능 저하 곡선이 완만하거나 거의 발생하지 않는다.
- **교차 스레드 자원 불균형 해결(노드 재분배):** 할당/반납 스레드가 달라도, ‘스레드 간 공용 공간’을 두어 넘치는 객체들을 청크 단위로 반납하기 때문에 서로 다른 풀 간의 균형이 유지된다.

3.5.4. 주요 적용 대상

멀티스레드 환경에서 생성과 소멸이 매우 빈번하게 일어나는 객체 전반에 적합하다.

- 락-프리 자료구조의 데이터 노드 (큐, 스택 등)
- 패킷(3.4.) (빈번하게 생성/소멸하는 직렬화 버퍼)
- 기타 라이프사이클이 짧은 멀티스레드 공용 객체 전반

4. 기타 특이사항

4.1. 네트워크 모듈 사용

4.1.1. 서버 시작

상속

네트워크 모듈은 클래스 형태로 제공된다. 어플리케이션은 이 클래스를 상속받아 프로젝트에 특화된 로직을 구현할 수 있다.

네트워크 모듈 클래스 생성자

```
NetLib_Server(  
    const WCHAR* openIP,  
    unsigned short openPort,  
    int opt_workerTH_pool_size,  
    int opt_concurrentTH_size,  
    int opt_maxOfSession,  
    bool opt_zerocpy,  
    Opt_Encryption* opt_encryption,  
    int opt_maxOfSendPackets);
```

openIP

리슨 소켓에 바인드 할 IP이다.

openPort

리슨 소켓에 바인드 할 Port이다.

opt_workerTH_pool_size

IOCP에서 사용할 워커 스레드의 총 개수이다.

opt_concurrentTH_size

IOCP에서 동시에 깨어날 수 있는 워커 스레드의 개수이다.

opt_workerTH_pool_size 이하로 설정할 수 있다.

opt_maxOfSession

네트워크 모듈에서 동시에 연결 가능한 세션의 개수이다. 65535까지 설정할 수 있다.

bool opt_zerocpy

네트워크 모듈에서 송신시 Zero_copy를 사용할지에 대한 여부이다.

opt_encryption

```
struct Opt_Encryption
{
    char Header_Code = 0x77;
    char Fixed_Key = 0x32;
};
```

XOR 기반 난독화에 사용되는 임의의 char 변수이다. 이 구조체는 클라이언트와 동일하게 설정해야 한다. nullptr로 설정하면 패킷 난독화 옵션이 사용되지 않는다.

예제 코드

[1] 네트워크 모듈 상속

```
class ChatServer:public NetLib_Server
{
public:
    ChatServer(
        const WCHAR* openIP,
        unsigned short openPort,
        int opt_workerTH_Pool_size,
        int opt_concurrentTH_size,
        int opt_maxOfSession,
        bool opt_zeroCopy,
        Opt_Encryption* opt_encryption);

    // 세션 최초 입장
    virtual void OnClientJoin(
        unsigned long long sessionHandle,
        unsigned long IP, unsigned short port
    );

    // 세션 해제
    virtual void OnClientLeave(
        unsigned long long sessionHandle,
        SESSION_LEAVE_CODE code,
        unsigned long IP,
        unsigned short port
    );

    // 연결 허용 여부
    virtual bool OnConnectionRequest(unsigned long IP, unsigned short port);
    /// 패킷 수신
    virtual void OnRecv(unsigned long long sessionHandle, Packet* packet);
};
```

[2] 네트워크 모듈 생성 및 시작

```
int main()
{
    Opt_Encryption encryption = {109, 30};

    server = new ChatServer(
        L"0.0.0.0",    // openIP
        6000,          // openPort
        16,            // opt_workerTH_Pool_size
        8,             // opt_concurrentTH_size
        20000,         // opt_maxOfSession
        true,          // opt_zerocopy
        &encryption    // opt_encryption
    );

    server->Start();
    printf("Server Start\n");
}
```

4.1.2. 가상 함수 이벤트



4.1.3. 패킷 수신

```
class ChatServer :public NetLib_Server
{
public:
    virtual void OnRecv(unsigned long long sessionHandle, Packet* packet)
    {
        PROFILING("CONTENT");

        WORD contentType;
        *packet >> contentType;
        Content_Proc(sessionHandle, contentType, packet); // 패킷 처리기
        InterlockedIncrement((long*)&update_tps);
    }
}
```

4.1.4. 패킷 송신

```
Packet* resPacket = Packet::Alloc();
*resPacket << (WORD)PACKET_CS_CHAT_RES_LOGIN << res_login << accNo;
server->SendPacket(sessionHandle, resPacket);
```

```
Packet::Free(resPacket);
```

4.2. 테스트 환경

4.1.1. 하드웨어 환경

4.1.1.1. 메인 서버

- 프로세서: Intel(R) Xeon(R) CPU E5-2680 v4 @ 2.40GHz, 2401Mhz, 14 코어, 28 논리 프로세서
- 메모리(RAM): 16.0GB
- NIC: Intel(R) I210 Gigabit Network Connection (더미와의 이더넷 핫라인)
Intel(R) I210 Gigabit Network Connection #2 (더미와의 이더넷 핫라인)
Realtek PCIe GBE Family Controller

4.1.1.2. 더미 서버 * 2

- 프로세서: Intel(R) Core(TM) i5-4460 CPU @ 3.20GHz, 3201Mhz, 4 코어, 4 논리 프로세서
- 메모리(RAM): 4.0GB
- NIC: Realtek PCIe GBE Family Controller (메인 서버와의 이더넷 핫라인)

4.1.2. 소프트웨어 환경

- OS 환경: Microsoft Windows Server 2019 10.0.17763
- OS 외 환경: (에코+에코개수(L7패킷18바이트), 세션수, 목표 송수신TPS, AcceptTPS,기간)
(Visual Studio, C++17, 컴파일 최적화 사용 안함, __inline만 확장)

4.1.3. 테스트 결과 요약

- 기간 2026.02.14 ~ 2026.02.21 (168시간)
- 대상 IOCP 네트워크 모듈 기반 채팅 서버

- 결과 요약

15000개 더미 세션의 반복적 재접속 상황에서
더미 다운 감지: 0건
패킷 에러 감지: 0건

- 회고

당시 실시간 모니터링을 통해 자원 누수 없음을 확인했으나 당시 테스트의 목적을 장기 가동 무결성 검증에 두어 최종 결과 데이터(스크린샷 및 로그)만 보존한게 아쉬움. 자동 백업 시스템의 필요성을 느낌.